BİL4010

Paralel programlama

11.Hafta

MPI ile kolektif haberleşme

İçerik

- MPI ile Kolektif Haberleşme
  - Senkronizasyon nasıl sağlanır
  - Broadcast fonksiyonu
  - Reduction fonksiyonu
  - Gather fonksiyonu
  - Scatter fonksiyonu

Kolektif Haberleşme Nedir?

- Kolektif haberleşme, bir grup işlemci(process) arasında verilerin gönderilmesini ve alınmasını kapsar.
- Genel olarak düşünüldüğünde, kolektif haberleşme rutinleri kullanılmaksızın yapılacak tüm işlemler MPI_Send ve MPI_Recv fonksiyonları kullanılarak yapılabilir.
- Fakat kolektif haberleşme fonksiyonları, bu işlemlerin çok daha kısa bir şekilde yapılmasını sağlar.
- Yani bir işlemci tek bir komut ile birden fazla işlemci ile aynı anda verileri paylaştırabilir ya da diğer işlemcilerden verileri toplayabilir.

Kolektif Haberleşme Faydaları

- Kolektif haberleşmenin kullanılması ile program içinde yazılacak MPI kodlarında önemli ölçüde bir azalma olur.
- Program geliştirilirken karşılaşılan hata sayısı azalır.
- Haberleşme rutinleri azalacağından program daha hızlı çalışabilir.

Kolektif Haberleşme Faydaları

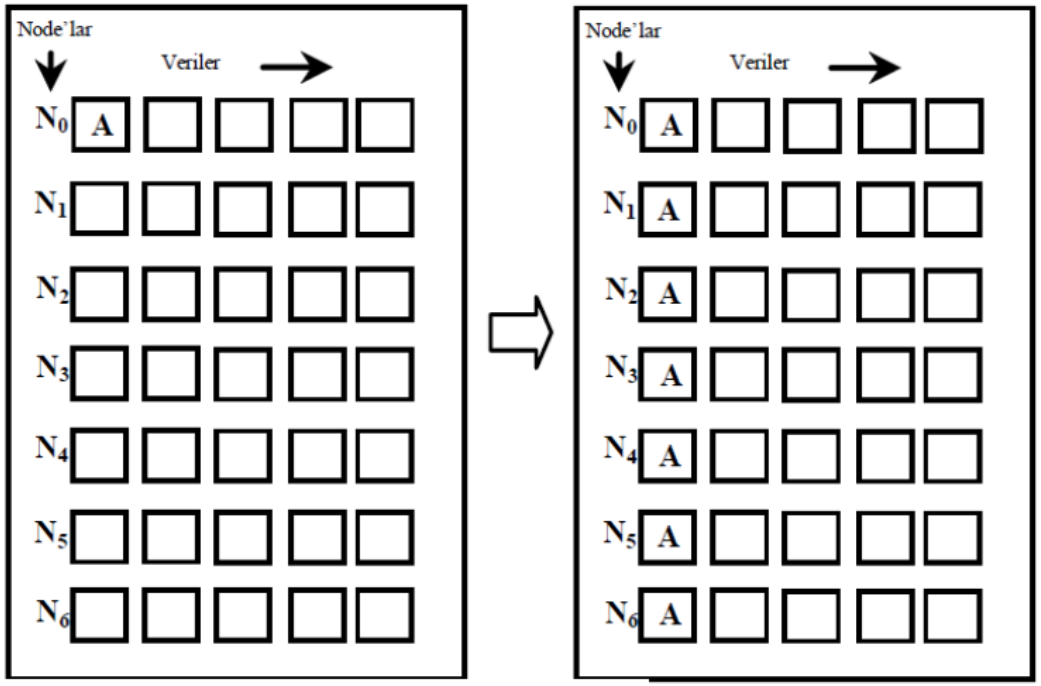
```
int MPI_Barrier(MPI_Comm    comm    /* in */);
```

- MPI_Barrier (MPI_COMM_WORLD);
- Bu fonksiyon işlemciler (process) arasındaki senkronizeliği sağlamak amacıyla kullanılır.
- Her bir işlemcinin herhangi bir işlem yapmadan önce diğer işlemcilerin belirlenen sürece kadar beklemesini sağlar.

Broadcast Fonksiyonu

- Broadcast fonksiyonu ile ana işlemcideki istenilen bir veri, haberleşme grubu içinde yer alan diğer tüm işlemcilere dağıtılır.

```
int MPI_Bcast(
        void*          data_p        /* in/out */,
        int            count         /* in     */,
        MPI_Datatype   datatype      /* in     */,
        int            source_proc   /* in     */,
        MPI_Comm       comm          /* in     */);
```



Ör: Broadcast Fonksiyonu

```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
}  /* Get_input */
```

Reduction Fonksiyonu (Reduce)

- Reduction fonksiyonu, her bir işlemciden istenilen verilerin alınarak temel toplam, çarpma, maksimum veya minimum değeri bulma gibi işlemlerin gerçekleştirilmesini sağlar.
- Sonuçlar ana işlemcide saklanır.

```
int MPI_Reduce(
        void*          input_data_p    /* in  */,
        void*          output_data_p   /* out */,
        int            count           /* in  */,
        MPI_Datatype   datatype        /* in  */,
        MPI_Op         operator        /* in  */,
        int            dest_process    /* in  */,
        MPI_Comm       comm            /* in  */);
```

Reduction Fonksiyonları

Operation Value	Meaning
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical and
MPI_BAND	Bitwise and
MPI_LOR	Logical or
MPI_BOR	Bitwise or
MPI_LXOR	Logical exclusive or
MPI_BXOR	Bitwise exclusive or
MPI_MAXLOC	Maximum and location of maximum
MPI_MINLOC	Minimum and location of minimum

Örn. MPI_Reduce

```
if (my_rank != 0) {
   MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
         MPI_COMM_WORLD);
} else {
   total_int = local_int;
   for (source = 1; source < comm_sz; source++) {
      MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);
      total_int += local_int;
   }
}
```

MPI Trap Kodu

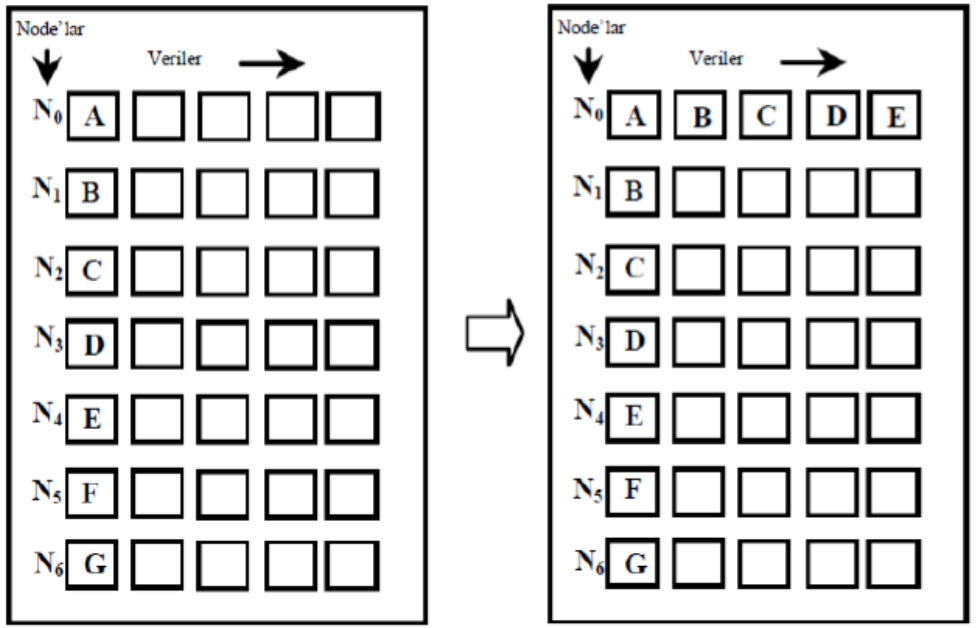
Bu kod ile yukarıdaki aynı işi yapıyor

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

Gather Fonksiyonu

- MPI_Gather fonksiyonu her bir işlemcideki verilerin master node'a kopyalanmasını sağlar.

```
int MPI_Gather(
        void*          send_buf_p   /* in   */,
        int            send_count   /* in   */,
        MPI_Datatype   send_type    /* in   */,
        void*          recv_buf_p   /* out  */,
        int            recv_count   /* in   */,
        MPI_Datatype   recv_type    /* in   */,
        int            dest_proc    /* in   */,
        MPI_Comm       comm         /* in   */);
```



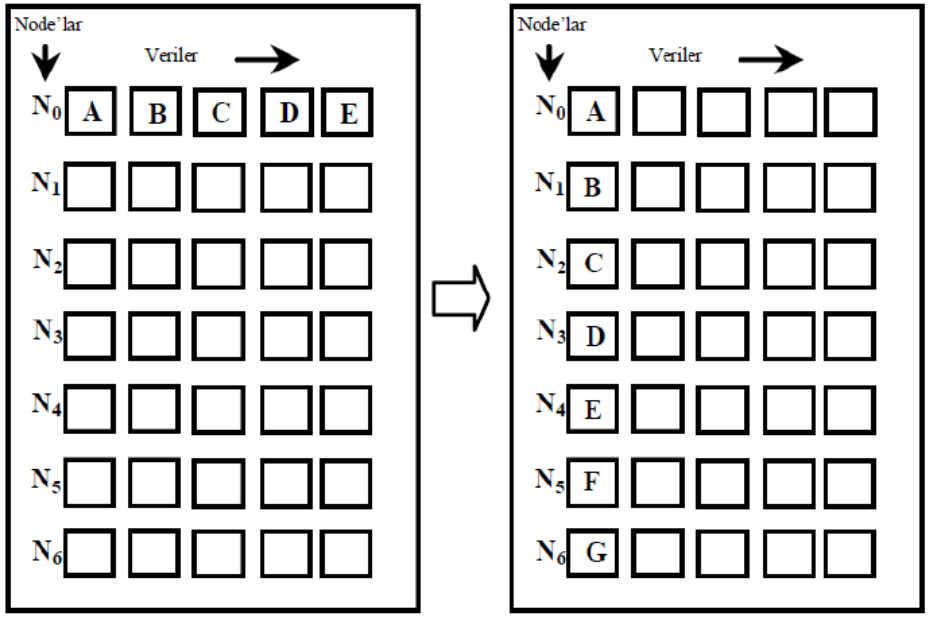
Örn: MPI_Gather

```
void Print_vector(
      double    local_b[]  /* in */,
      int       local_n    /* in */,
      int       n          /* in */,
      char      title[]    /* in */,
      int       my_rank    /* in */,
      MPI_Comm  comm       /* in */) {

   double* b = NULL;
   int i;

   if (my_rank == 0) {
      b = malloc(n*sizeof(double));
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n,
            MPI_DOUBLE, 0, comm);
      printf("%s\n", title);
      for (i = 0; i < n; i++)
         printf("%f ", b[i]);
      printf("\n");
      free(b);
   } else {
      MPI_Gather(local_b, local_n, MPI_DOUBLE, b, local_n,
            MPI_DOUBLE, 0, comm);
   }
}  /* Print_vector */
```

Scatter Fonksiyonu

- MPI_Scatter fonksiyon, ana işlemcide bulunan dizi elemanlarının sırası ile diğer işlemcilere dağıtılması amacıyla kullanılır.

```
int MPI_Scatter(
      void*          send_buf_p   /* in  */,
      int            send_count   /* in  */,
      MPI_Datatype   send_type    /* in  */,
      void*          recv_buf_p   /* out */,
      int            recv_count   /* in  */,
      MPI_Datatype   recv_type    /* in  */,
      int            src_proc     /* in  */,
      MPI_Comm       comm         /* in  */);
```



Örn: MPI_Scatter

```
void Read_vector(
      double    local_a[]   /* out */,
      int       local_n     /* in  */,
      int       n           /* in  */,
      char      vec_name[]  /* in  */,
      int       my_rank     /* in  */,
      MPI_Comm  comm        /* in  */) {

   double* a = NULL;
   int i;

   if (my_rank == 0) {
      a = malloc(n*sizeof(double));
      printf("Enter the vector %s\n", vec_name);
      for (i = 0; i < n; i++)
         scanf("%lf", &a[i]);
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n,
            MPI_DOUBLE, 0, comm);
      free(a);
   } else {
      MPI_Scatter(a, local_n, MPI_DOUBLE, local_a, local_n,
            MPI_DOUBLE, 0, comm);
   }
}  /* Read_vector */
```

Teşekkürler.

Sorular...